Arquivo: Dashboard.js

text import React, { useEffect, useState } from ‘react’; import { useNavigate } from ‘react-router-dom’; import { fetchTransactions, fetchSaldo, deleteTransaction, setupAxiosInterceptors } from ‘../api/transactions’; import TransactionForm from ‘../components/TransactionForm’; import TransactionList from ‘../components/TransactionList’; import EditTransactionModal from ‘../components/EditTransactionModal’; import SaldoIndicator from ‘../components/SaldoIndicator’; import ConfirmationModal from ‘../components/ConfirmationModal’; import Notification from ‘../components/Notification’; import Navbar from ‘../components/Navbar’; import { jsPDF } from ‘jspdf’; import autoTable from ‘jspdf-autotable’; import { motion, AnimatePresence } from ‘framer-motion’; import axiosLocal from ‘axios’; import logo from ‘../assets/logo.jpg’; import ‘./Dashboard.css’; import { getTipoDisplay, getCultoDisplay, getTipoDespesaDisplay, formatDate } from ‘../utils’; import { FaCalendarAlt, FaSearch } from ‘react-icons/fa’;

const Dashboard = () => { const [selectedMonth, setSelectedMonth] = useState(”); const [selectedYear, setSelectedYear] = useState(new Date().getFullYear()); const [selectedTransaction, setSelectedTransaction] = useState(null); const [showForm, setShowForm] = useState(false); const [transactions, setTransactions] = useState([]); const [filteredTransactions, setFilteredTransactions] = useState([]); const [editingTransaction, setEditingTransaction] = useState(null); const [saldo, setSaldo] = useState(0); const [error, setError] = useState(null); const [isLoading, setIsLoading] = useState(false); const [isModalOpen, setIsModalOpen] = useState(false); const [transactionToDelete, setTransactionToDelete] = useState(null); const [notification, setNotification] = useState(null); const [showFilters, setShowFilters] = useState(true); const [showReportOptions, setShowReportOptions] = useState(false); const [includeGratificacao, setIncludeGratificacao] = useState(true); const [includeDizimoGratificacao, setIncludeDizimoGratificacao] = useState(true); const [includeDizimoIgreja, setIncludeDizimoIgreja] = useState(true); const [reportTransactions, setReportTransactions] = useState([]);

const [searchDay, setSearchDay] = useState(’ ‘); const [searchDescription, setSearchDescription] = useState(‘‘); const [searchType, setSearchType] = useState(’ ’);

const [currentPage, setCurrentPage] = useState(1); const transactionsPerPage = 10;

const navigate = useNavigate();

useEffect(() => { setupAxiosInterceptors(navigate); }, [navigate]);

const updateTransactions = async () => { try { const transData = await fetchTransactions(selectedMonth || null, selectedYear, navigate); setTransactions(transData); setFilteredTransactions(transData); setCurrentPage(1);

} catch (err) { setNotification({ message: ‘Erro ao atualizar transações:’ + err.message, type: ‘error’ }); } };

const updateSaldo = async () => { try { const saldoData = await fetchSaldo(navigate); setSaldo(saldoData); } catch (err) { setNotification({ message: ‘Erro ao atualizar saldo:’ + err.message, type: ‘error’ }); } };

const fetchData = async () => { setIsLoading(true); setError(null); try { const [transData, saldoData] = await Promise.all([fetchTransactions(selectedMonth || null, selectedYear, navigate), fetchSaldo(navigate),]); setTransactions(transData); setFilteredTransactions(transData); setSaldo(saldoData); setCurrentPage(1); } catch (err) { setError(err.message); } finally { setIsLoading(false); } };

useEffect(() => { fetchData(); }, [selectedMonth, selectedYear]);

useEffect(() => { let filtered = transactions;

```
if (searchDay) {
  filtered = filtered.filter((transaction) => {
    const day = new Date(transaction.data).toLocaleDateString('pt-BR', { day: '2-digit' });
    return day === searchDay;
  });
}

if (searchDescription) {
  filtered = filtered.filter((transaction) =>
    (transaction.descricao || '').toLowerCase().includes(searchDescription.toLowerCase())
  );
}

if (searchType) {
  filtered = filtered.filter((transaction) => transaction.tipo === searchType);
}

setFilteredTransactions(filtered);
setCurrentPage(1);
```

}, [searchDay, searchDescription, searchType, transactions]);

const handleTransactionAdded = () => { updateTransactions(); updateSaldo(); };

const handleDelete = (transaction) => { setTransactionToDelete(transaction); setIsModalOpen(true); };

const confirmDelete = async () => { try { await deleteTransaction(transactionToDelete.id, navigate); setNotification({ message: ‘Transação excluída com sucesso!’, type:

‘success’ }); setSelectedTransaction(null); updateTransactions(); updateSaldo(); setIsModalOpen(false); setTransactionToDelete(null); } catch (err) { setNotification({ message: ‘Erro ao excluir transação:’ + err.message, type: ‘error’ }); setIsModalOpen(false); } };

const handleEdit = (transaction) => { setSelectedTransaction(null); setEditingTransaction(transaction); };

const handleSave = () => { setNotification({ message: ‘Transação editada com sucesso!’, type: ‘success’ }); updateTransactions(); updateSaldo(); setEditingTransaction(null); };

const indexOfLastTransaction = currentPage * transactionsPerPage; const indexOfFirstTransaction = indexOfLastTransaction - transactionsPerPage; const currentTransactions = filteredTransactions.slice(indexOfFirstTransaction, indexOfLastTransaction); const totalPages = Math.ceil(filteredTransactions.length / transactionsPerPage);

const handlePreviousPage = () => { if (currentPage > 1) { setCurrentPage(currentPage - 1); } };

const handleNextPage = () => { if (currentPage < totalPages) { setCurrentPage(currentPage + 1); } };

const handlePageClick = (pageNumber) => { setCurrentPage(pageNumber); };

const generateMonthlyReport = async () => { setShowReportOptions(false); setIsLoading(true); setError(null);

```
try {
  const transactions = reportTransactions;

  transactions.sort((a, b) => new Date(a.data) - new Date(b.data));

  const reportData = {};
  transactions.forEach((transaction) => {
    const date = new Date(transaction.data);
    const day = date.toLocaleDateString('pt-BR', { day: '2-digit' });

    if (!reportData[day]) {
      reportData[day] = {
        dizimos: 0,
        ofertas: 0,
        despesas: [],
      };
    }

    if (transaction.tipo === 'D') {
```

```
    reportData[day].dizimos += parseFloat(transaction.quantia);
  } else if (transaction.tipo === 'O') {
    reportData[day].ofertas += parseFloat(transaction.quantia);
  } else if (transaction.tipo === 'S') {
    let discriminacaoText = `${getTipoDespesaDisplay(transaction.tipo_despesa)}${transacti
    reportData[day].despesas.push({
      quantia: parseFloat(transaction.quantia),
      discriminacao: discriminacaoText,
    });
  }
});

const formattedData = [];
Object.keys(reportData).forEach((day) => {
  if (reportData[day].dizimos > 0) {
    formattedData.push({
      dia: day,
      discriminacao: 'Dízimo',
      entrada: reportData[day].dizimos.toFixed(2),
      saida: '0.00',
    });
  }

  if (reportData[day].ofertas > 0) {
    formattedData.push({
      dia: day,
      discriminacao: 'Oferta',
      entrada: reportData[day].ofertas.toFixed(2),
      saida: '0.00',
    });
  }

  reportData[day].despesas.forEach((despesa) => {
    formattedData.push({
      dia: day,
      discriminacao: despesa.discriminacao,
      entrada: '0.00',
      saida: despesa.quantia.toFixed(2),
    });
  });
});

let totalEntradas = transactions
  .filter((t) => t.tipo === 'D' || t.tipo === 'O')
  .reduce((sum, t) => sum + parseFloat(t.quantia), 0);
let totalSaidas = transactions
```

```
  .filter((t) => t.tipo === 'S')
  .reduce((sum, t) => sum + parseFloat(t.quantia), 0);
let saldoMes = totalEntradas - totalSaidas;

const gratificacaoPastor = 900;
let gratificacaoAplicada = false;
if (includeGratificacao && saldoMes >= gratificacaoPastor) {
  formattedData.push({
    dia: '',
    discriminacao: 'Gratificação do Pastor',
    entrada: '0.00',
    saida: gratificacaoPastor.toFixed(2),
  });
  totalSaidas += gratificacaoPastor;
  saldoMes -= gratificacaoPastor;
  gratificacaoAplicada = true;
}

const dizimoGratificacao = gratificacaoPastor * 0.1;
if (gratificacaoAplicada && includeDizimoGratificacao) {
  formattedData.push({
    dia: '',
    discriminacao: 'Dízimo da Gratificação',
    entrada: dizimoGratificacao.toFixed(2),
    saida: '0.00',
  });
  totalEntradas += dizimoGratificacao;
  saldoMes += dizimoGratificacao;
}

if (includeDizimoIgreja) {
  const dizimoIgreja = totalEntradas * 0.1;
  formattedData.push({
    dia: '',
    discriminacao: 'Dízimo da Igreja',
    entrada: '0.00',
    saida: dizimoIgreja.toFixed(2),
  });
  totalSaidas += dizimoIgreja;
  saldoMes -= dizimoIgreja;
}

for (let i = 0; i < 5; i++) {
  formattedData.push({
    dia: '',
    discriminacao: '',
```

```
    entrada: '',
    saida: '',
  });
}

const previousMonth = selectedMonth === 1 ? 12 : selectedMonth - 1;
const previousYear = selectedMonth === 1 ? selectedYear - 1 : selectedYear;
const axios = axiosLocal.create({
  baseURL: process.env.REACT_APP_API_URL,
  headers: {
    Authorization: `Bearer ${localStorage.getItem('token')}`,
  },
});
const previousResponse = await axios.get(`/api/transacoes/?ano__lte=${previousYear}&mes__l
const previousTransactions = previousResponse.data;
const previousEntradas = previousTransactions
  .filter((t) => t.tipo === 'D' || t.tipo === 'O')
  .reduce((sum, t) => sum + parseFloat(t.quantia), 0);
const previousSaidas = previousTransactions
  .filter((t) => t.tipo === 'S')
  .reduce((sum, t) => sum + parseFloat(t.quantia), 0);
const saldoAnterior = previousEntradas - previousSaidas;
const totalCaixa = saldoAnterior + saldoMes;

const doc = new jsPDF();
doc.setFont("times");

try {
  doc.addImage(logo, 'JPG', 14, 10, 30, 30);
} catch (err) {
  console.error('Erro ao adicionar o logotipo:', err);
  setNotification({ message: 'Erro ao adicionar o logotipo ao PDF. Verifique o arquivo da
}

doc.setFontSize(14);
doc.setFont("times", "bold");
doc.text('IGREJA DE DEUS MISSIONÁRIA', 105, 20, { align: 'center' });
doc.setFontSize(10);
doc.setFont("times", "normal");
doc.text('CNPJ: 05.869.914/0001-07', 105, 28, { align: 'center' });
doc.text('DEPARTAMENTO FINANCEIRO', 105, 36, { align: 'center' });
doc.setFontSize(12);
doc.text(`MÊS: ${new Date(0, selectedMonth - 1).toLocaleString('pt-BR', { month: 'long' })
doc.text(`ANO: ${selectedYear}`, 105, 50, { align: 'center' });
doc.text('EBENÉZER', 180, 50);
```

```
autoTable(doc, {
  startY: 60,
  head: [['DIA', 'DISCRIMINAÇÃO', 'ENTRADA', 'SAÍDA']],
  body: formattedData.map((row) => [
    row.dia,
    row.discriminacao,
    row.entrada === '0.00' || row.entrada === '' ? '-' : `R$ ${row.entrada}`,
    row.saida === '0.00' || row.saida === '' ? '-' : `R$ ${row.saida}`,
  ]),
  theme: 'grid',
  headStyles: {
    fillColor: [255, 255, 255],
    textColor: [0, 0, 0],
    fontStyle: 'bold',
    font: 'times',
    lineWidth: 0.2,
    lineColor: [0, 0, 0],
  },
  styles: {
    fontSize: 10,
    cellPadding: 2,
    font: 'times',
    textColor: [0, 0, 0],
    lineWidth: 0.2,
    lineColor: [0, 0, 0],
  },
  columnStyles: {
    0: { cellWidth: 20 },
    1: { cellWidth: 100 },
    2: { cellWidth: 35, halign: 'right' },
    3: { cellWidth: 35, halign: 'right' },
  },
});

const finalY = doc.lastAutoTable.finalY + 15;
doc.setFontSize(12);
doc.setFont("times", "bold");
doc.text(`TOTAL DE ENTRADA: R$ ${totalEntradas.toFixed(2)}`, 14, finalY);
doc.text(`TOTAL DE SAÍDA DO MÊS: R$ ${totalSaidas.toFixed(2)}`, 14, finalY + 8);
doc.text(`SALDO DO MÊS: R$ ${saldoMes.toFixed(2)}`, 14, finalY + 16);
doc.text(`SALDO ANTERIOR: R$ ${saldoAnterior.toFixed(2)}`, 14, finalY + 24);
doc.text(`TOTAL EM CAIXA: R$ ${totalCaixa.toFixed(2)}`, 14, finalY + 32);

const signatureY = finalY + 50;
doc.setFontSize(10);
doc.setFont("times", "normal");
```

```
  doc.text('TESOUREIRO: _____', 14, signatureY);
  doc.text('DIRIGENTE DA CONGREGAÇÃO: _____', 14, signatureY + 10);
  doc.text('DIRETOR FINANCEIRO IDM SEDE: Patrícia Maria de Silva', 14, signatureY + 20);
  doc.text('CONSELHO FISCAL: _____', 14, signatureY + 30);

  doc.save(`relatorio_financeiro_${selectedMonth}_${selectedYear}.pdf`);
  setNotification({ message: 'Relatório gerado com sucesso!', type: 'success' });
} catch (err) {
  setNotification({ message: 'Erro ao gerar o relatório: ' + err.message, type: 'error' });
} finally {
  setIsLoading(false);
}

};
```

const openReportOptions = async () => { if (!selectedMonth || !selectedYear) { setNotification({ message: 'Por favor, selecione um mês e ano para gerar o relatório.', type: 'error' }); return; }

```
try {
  const axios = axiosLocal.create({
    baseURL: process.env.REACT_APP_API_URL,
    headers: {
      Authorization: `Bearer ${localStorage.getItem('token')}`,
    },
  });

  const response = await axios.get(`/api/transacoes/?mes=${selectedMonth}&ano=${selectedYear
  const transactions = response.data;

  setReportTransactions(transactions);
  setShowReportOptions(true);
} catch (err) {
  setNotification({ message: 'Erro ao buscar transações para o relatório: ' + err.message, t
}

};
```

return (<motion.div className="dashboard" initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ duration: 0.5 }}>

```
  <div className="dashboard-content">
    {isLoading && (
      <motion.div
        className="loading"
        initial={{ opacity: 0 }}
```

```
    animate={{ opacity: 1 }}
    transition={{ duration: 0.3 }}
  >
    Carregando...
  </motion.div>
)}
{error && (
  <motion.div
    className="error-message"
    initial={{ opacity: 0, scale: 0.9 }}
    animate={{ opacity: 1, scale: 1 }}
    transition={{ duration: 0.3 }}
  >
    {error}
    <button onClick={fetchData}>Tentar novamente</button>
  </motion.div>
)}
{!isLoading && !error && (
  <>
    <SaldoIndicator className="saldo-indicator" saldo={saldo} />

    <motion.button
      className="dashboard-button"
      onClick={() => setShowForm(!showForm)}
      whileHover={{ scale: 1.05 }}
      whileTap={{ scale: 0.95 }}
    >
      {showForm ? 'Fechar Formulário' : 'Criar/Editar/Deletar Registro'}
    </motion.button>

    {showForm && (
      <motion.div
        className="form-container"
        initial={{ opacity: 0, height: 0 }}
        animate={{ opacity: 1, height: 'auto' }}
        exit={{ opacity: 0, height: 0 }}
        transition={{ duration: 0.3 }}
      >
        <TransactionForm onTransactionAdded={handleTransactionAdded} setNotification={se
      </motion.div>
    )}

    <div className="month-filter">
      <select
        value={selectedMonth}
        onChange={(e) => setSelectedMonth(e.target.value === '' ? '' : Number(e.target.v
```

```
  >
    <option value="">Todos os Meses</option>
    {Array.from({ length: 12 }, (_, i) => (
      <option key={i + 1} value={i + 1}>
        {new Date(0, i).toLocaleString('pt-BR', { month: 'long' })}
      </option>
    ))}
  </select>
  <input
    type="number"
    value={selectedYear}
    onChange={(e) => setSelectedYear(Number(e.target.value))}
    min="2020"
    max={new Date().getFullYear()}
  />
  <motion.button
    className="dashboard-button"
    onClick={openReportOptions}
    whileHover={{ scale: 1.05 }}
    whileTap={{ scale: 0.95 }}
  >
    Gerar Relatório Mensal
  </motion.button>
</div>

<motion.button
  className="filter-toggle-button"
  onClick={() => setShowFilters(!showFilters)}
  whileHover={{ scale: 1.05 }}
  whileTap={{ scale: 0.95 }}
>
  {showFilters ? 'Esconder Filtros' : 'Mostrar Filtros'}
</motion.button>

<AnimatePresence>
  {showFilters && (
    <motion.div
      className="search-filters"
      initial={{ opacity: 0, height: 0 }}
      animate={{ opacity: 1, height: 'auto' }}
      exit={{ opacity: 0, height: 0 }}
      transition={{ duration: 0.3 }}
    >
      <div className="filter-group">
        <div className="filter-item">
          <label>Dia</label>
```

```
            <div className="input-container">
              <FaCalendarAlt />
              <input
                type="text"
                value={searchDay}
                onChange={(e) => setSearchDay(e.target.value)}
              />
            </div>
          </div>
          <div className="filter-item">
            <label>Descrição</label>
            <div className="input-container">
              <FaSearch />
              <input
                type="text"
                value={searchDescription}
                onChange={(e) => setSearchDescription(e.target.value)}
              />
            </div>
          </div>
          <div className="filter-item">
            <label>Tipo</label>
            <select value={searchType} onChange={(e) => setSearchType(e.target.value)}
              <option value="">Todos os Tipos</option>
              <option value="D">Dízimo</option>
              <option value="O">Oferta</option>
              <option value="S">Despesa</option>
            </select>
          </div>
        </div>
      </motion.div>
    )}
  </AnimatePresence>

  <div className="transaction-list">
    <TransactionList
      transactions={currentTransactions}
      onEdit={handleEdit}
      onDelete={handleDelete}
      onTransactionClick={setSelectedTransaction}
      setTransactions={setTransactions}
      setNotification={setNotification}
    />
  </div>

  {totalPages > 1 && (
```

```
  <div className="pagination">
    <motion.button
      onClick={handlePreviousPage}
      disabled={currentPage === 1}
      whileHover={{ scale: 1.05 }}
      whileTap={{ scale: 0.95 }}
    >
      Anterior
    </motion.button>

    {Array.from({ length: totalPages }, (_, index) => index + 1).map((pageNumber) =>
      <motion.button
        key={pageNumber}
        onClick={() => handlePageClick(pageNumber)}
        className={currentPage === pageNumber ? 'active' : ''}
        whileHover={{ scale: 1.05 }}
        whileTap={{ scale: 0.95 }}
      >
        {pageNumber}
      </motion.button>
    ))}

    <motion.button
      onClick={handleNextPage}
      disabled={currentPage === totalPages}
      whileHover={{ scale: 1.05 }}
      whileTap={{ scale: 0.95 }}
    >
      Próximo
    </motion.button>
  </div>
)}

{editingTransaction && (
  <EditTransactionModal
    transaction={editingTransaction}
    onClose={() => setEditingTransaction(null)}
    onSave={handleSave}
    setNotification={setNotification}
  />
)}
{selectedTransaction && (
  <motion.div
    className="modal-overlay"
    initial={{ opacity: 0 }}
    animate={{ opacity: 1 }}
```

```
  exit={{ opacity: 0 }}
  transition={{ duration: 0.3 }}
  onClick={(e) => e.target === e.currentTarget && setSelectedTransaction(null)}
>
  <motion.div
    className="modal-content"
    initial={{ scale: 0.9, opacity: 0 }}
    animate={{ scale: 1, opacity: 1 }}
    exit={{ scale: 0.9, opacity: 0 }}
    transition={{ duration: 0.3 }}
  >
    <h3>Detalhes da Transação</h3>
    <p>
      <strong>Tipo:</strong> {getTipoDisplay(selectedTransaction.tipo)}
    </p>
    <p>
      <strong>Valor:</strong> R$ {selectedTransaction.quantia}
    </p>
    <p>
      <strong>Data:</strong> {formatDate(selectedTransaction.data)}
    </p>
    {selectedTransaction.tipo === 'D' && (
      <p>
        <strong>Contribuinte:</strong> {selectedTransaction.nome || '-'}
      </p>
    )}
    {selectedTransaction.tipo === 'O' && (
      <p>
        <strong>Culto:</strong> {getCultoDisplay(selectedTransaction.culto) || '-
      </p>
    )}
    {selectedTransaction.tipo === 'S' && (
      <p>
        <strong>Tipo de Despesa:</strong> {getTipoDespesaDisplay(selectedTransacti
      </p>
    )}
    <p>
      <strong>Descrição:</strong> {selectedTransaction.descricao || '-'}
    </p>
    <div className="button-group">
      <button className="edit" onClick={() => handleEdit(selectedTransaction)}>
        Editar
      </button>
      <button className="delete" onClick={() => handleDelete(selectedTransaction)]
        Excluir
      </button>
```

```
        <button className="close" onClick={() => setSelectedTransaction(null)}>
          Fechar
        </button>
      </div>
    </motion.div>
  </motion.div>
)}
<ConfirmationModal
  isOpen={isModalOpen}
  onClose={() => setIsModalOpen(false)}
  onConfirm={confirmDelete}
  message="Tem certeza que deseja excluir esta transação?"
/>

{/* Modal de Opções do Relatório */}
{showReportOptions && (
  <motion.div
    className="modal-overlay"
    initial={{ opacity: 0 }}
    animate={{ opacity: 1 }}
    exit={{ opacity: 0 }}
    transition={{ duration: 0.3 }}
    onClick={(e) => e.target === e.currentTarget && setShowReportOptions(false)}
  >
    <motion.div
      className="modal-content"
      initial={{ scale: 0.9, opacity: 0 }}
      animate={{ scale: 1, opacity: 1 }}
      exit={{ scale: 0.9, opacity: 0 }}
      transition={{ duration: 0.3 }}
    >
      <h3>Opções do Relatório</h3>
      <div style={{ marginBottom: '15px' }}>
        <label style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
          <input
            type="checkbox"
            checked={includeGratificacao}
            onChange={(e) => setIncludeGratificacao(e.target.checked)}
          />
          Incluir Gratificação do Pastor (R$ 900,00)
        </label>
      </div>
      <div style={{ marginBottom: '15px' }}>
        <label style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
          <input
            type="checkbox"
```

```
                    checked={includeDizimoGratificacao}
                    onChange={(e) => setIncludeDizimoGratificacao(e.target.checked)}
                    disabled={!includeGratificacao}
                  />
                  Incluir Dízimo da Gratificação (10% da gratificação)
                </label>
              </div>
              <div style={{ marginBottom: '15px' }}>
                <label style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
                  <input
                    type="checkbox"
                    checked={includeDizimoIgreja}
                    onChange={(e) => setIncludeDizimoIgreja(e.target.checked)}
                  />
                  Incluir Dízimo da Igreja (10% das entradas)
                </label>
              </div>
              <div className="button-group">
                <button className="edit" onClick={generateMonthlyReport}>
                  Gerar Relatório
                </button>
                <button className="close" onClick={() => setShowReportOptions(false)}>
                  Cancelar
                </button>
              </div>
            </motion.div>
          </motion.div>
        )}
      </>
    )}
    {notification && (
      <Notification
        message={notification.message}
        type={notification.type}
        onClose={() => setNotification(null)}
      />
    )}
  </div>
</motion.div>

); };
export default Dashboard;
```